SHARDA UNIVERSITY
Beyond Boundaries

B.Tech Project Evaluation-2 ,VIIth Sem

Project Title - Intelligent DevOps Monitoring & Incident Response System

Presented by :

Muhammad Shayaan Ali – 2023433557
Riddhima Sharma – 2023567320
Shruti Tyagi – 2023566902

Under the Supervision of:-

Ms. Aliza Rana,
Assistant Professor, CSE

DEPARTMENT OF COMPUTER SCIENCE & ENGINEERING
SHARDA SCHOOL OF ENGINEERING AND TECHNOLOGY

August 2026

Contents of the Presentation

- Title of the Project
- Introduction to the Project
- Motivation for the Project
- Problem Statement / Objectives of the Project
- Problem Domain
- Existing Systems
- Team Members & Workload Distribution
- Meeting Details
- Project Planning / Timeline
- Current Status & Results
- Uniqueness of the Idea
- Expected Outcome
- Conclusion

Introduction to the Project

Intelligent DevOps Monitoring System:
The Intelligent DevOps Monitoring & Incident Response System is an automated platform designed to monitor applications and infrastructure, identify abnormal behavior, analyze incidents and assist in resolving them.

The system combines:

- Real-Time Monitoring
  Prometheus collects application, container and dependency metrics every 5 seconds.
- Hybrid Anomaly Detection
  Alert rules catch known failures; an ML detector catches unusual behavior.
- LLM Incident Analysis
  An LLM finds the root-cause service and proposes a fix; YAML runbooks are the fallback.
- Safe Automated Response
  A policy validator checks every action before it runs; risky actions need human approval.
- Verification & Escalation
  Each fix is verified; if the problem persists it is re-analyzed (max 3 attempts), then escalated.

Key Idea
Instead of only generating alerts, the system turns monitoring data into a root-cause diagnosis and a safe, verified response.

Motivation for the Project

Why do we need this?
Modern applications generate large amounts of operational data through:

- Server Metrics
- Request Rates, Latency & Errors
- CPU & Memory Usage
- Network Metrics
- Container Metrics
- Service Health Data

However, this information often becomes:

- Difficult to analyze manually
- Too large to monitor continuously
- Difficult to correlate
- Responsible for alert fatigue
- Slow to convert into useful actions

Motivation:
We wanted to build a system where monitoring data can be continuously collected, intelligently analyzed and converted into actionable incident responses.

Our Goal:
Move from:
“An alert has occurred.” to
“An issue has been detected, analyzed and an appropriate response has been identified.”

Problem Statement / Objectives

Problem Statement:
Traditional monitoring systems primarily collect metrics and generate alerts, but they often require engineers to manually analyze incidents, identify root causes and execute corrective actions.

There is a need for a unified system that provides:
Monitoring + Anomaly Detection + AI Analysis + Safe, Verified Incident Response
while maintaining reliability and operational control.

Objectives:
- O1 Monitor all services and containers – 100% of services instrumented.
- O2 Detect faults (rules + ML) – recall ≥ 90%, false positives ≤ 10%.
- O3 Identify the root-cause service and fault type – ≥ 80% accuracy.
- O4 Recommend a safe, correct action – ≥ 85% correct, 0 unsafe actions executed.
- O5 Resolve faults without human help – ≥ 60% auto-resolved and verified.
- O6 Reduce mean time to resolve (MTTR) – ≥ 50% faster than fixing by hand.

Problem Domain

DevOps Monitoring + AIOps + Artificial Intelligence

The project lies at the intersection of:

- DevOps Monitoring:-Continuous monitoring of applications, services and infrastructure.
- Anomaly Detection:-Identifying unusual patterns in system metrics and behavior.
- Artificial Intelligence:-Using LLM-based reasoning to interpret incidents and generate useful responses.
- Incident Management:-Identifying, prioritizing and tracking infrastructure incidents.
- Automation:-Executing predefined actions to reduce manual intervention.
- Distributed Systems:-The platform integrates multiple components for monitoring, communication, AI analysis and response execution.

Existing Systems

Existing System	Strength	Limitation
Prometheus	Powerful time-series monitoring and metric collection	Primarily focuses on monitoring; limited intelligent incident diagnosis and automated remediation
Grafana	Excellent visualization, dashboards and alert monitoring	Requires external systems for deeper incident analysis and automated response
ELK Stack	Strong centralized log collection and log analysis	Large-scale setup can be complex; limited autonomous incident resolution
Datadog	Integrated monitoring, logs, traces and alerting	Primarily relies on predefined monitoring and alerting workflows; limited customization for project-specific autonomous remediation
Our Platform	Monitoring + LLM-based diagnosis + incident detection + intelligent response/remediation	Requires development and testing of a reliable test bench and response mechanism

Research Gap

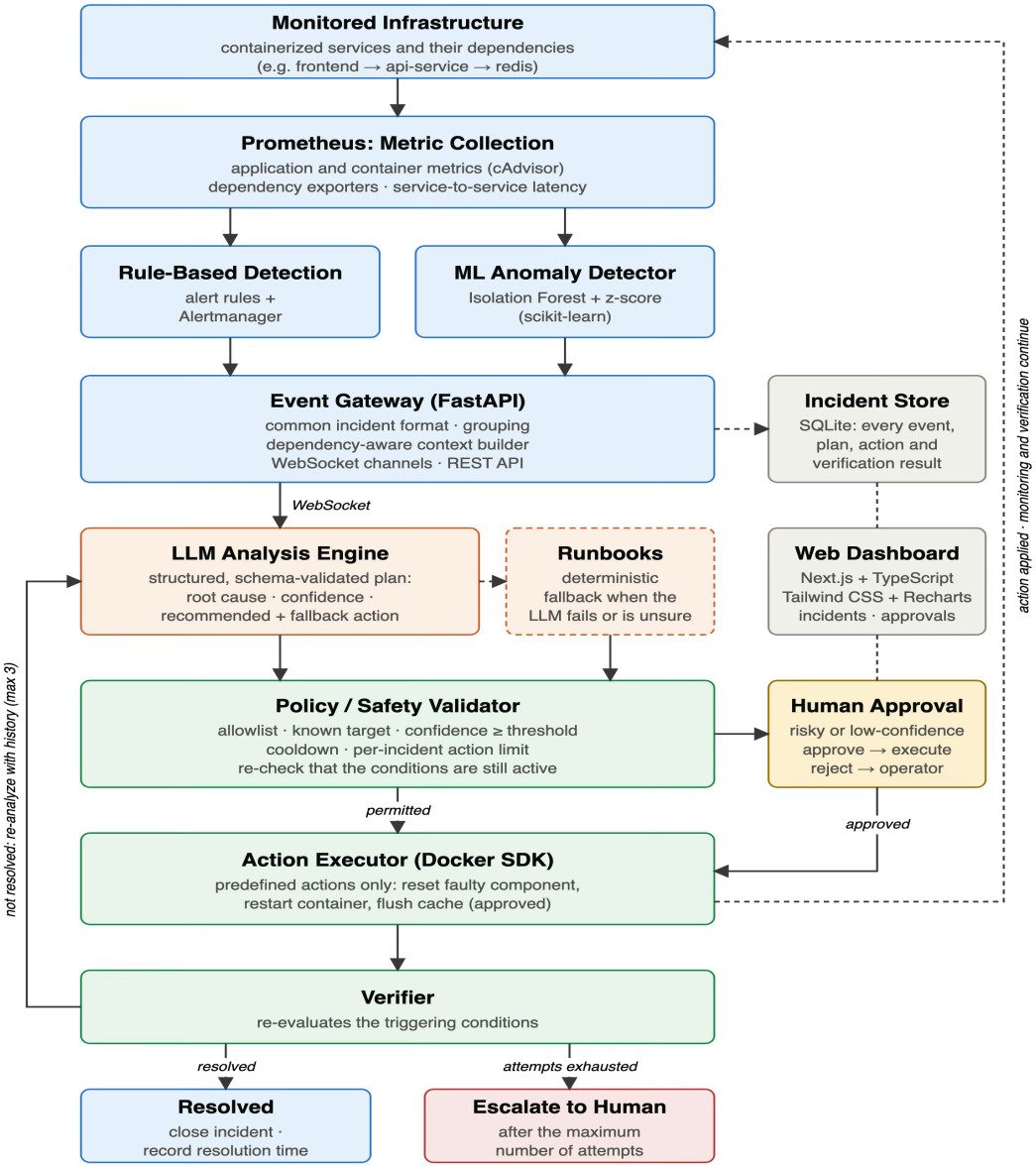
Most existing DevOps solutions primarily focus on:

Monitoring & Visualization or **Alerting & Incident Management or Observability**

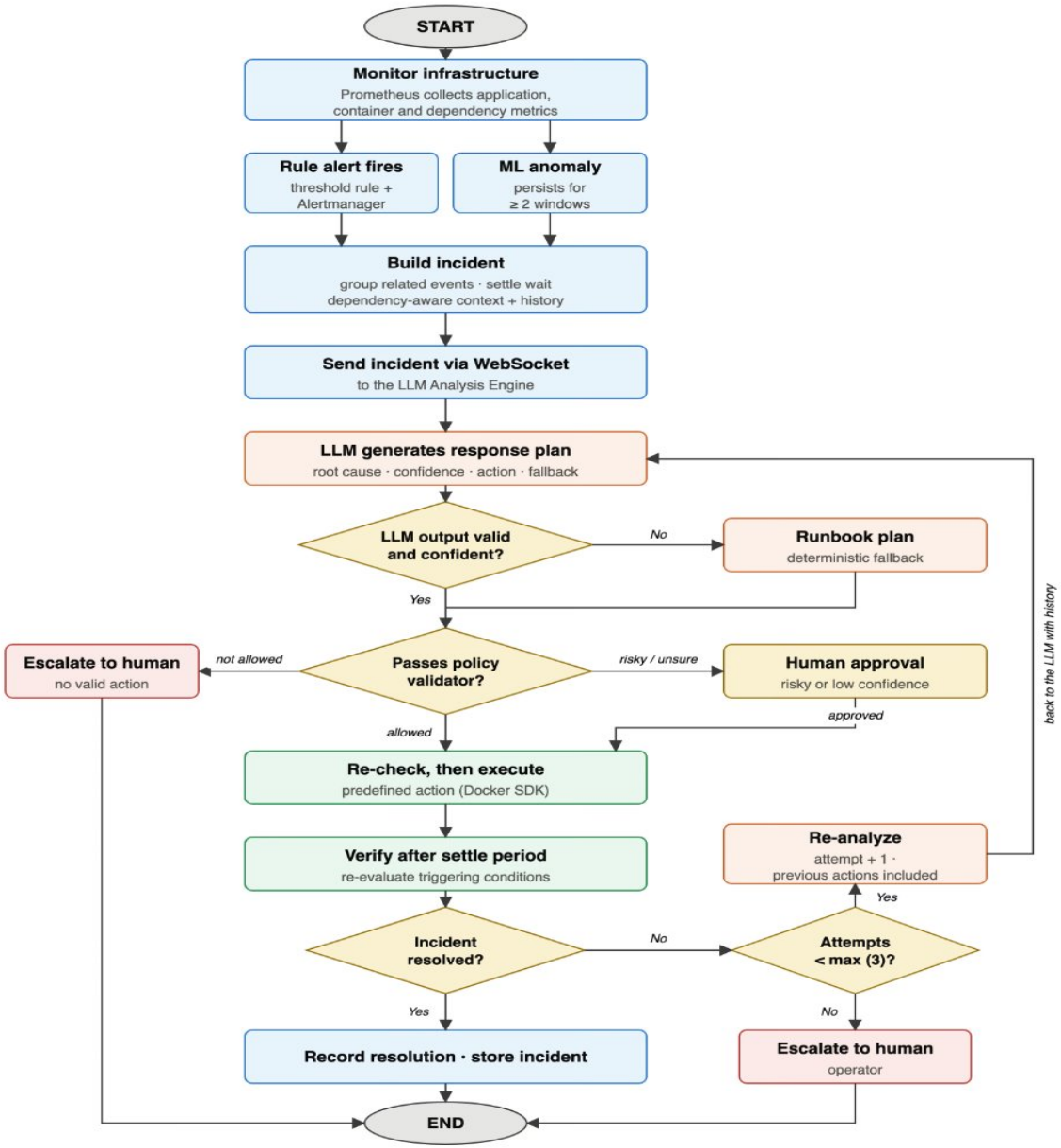
Our project aims to combine:

Real-time Monitoring + Incident Detection + LLM-based Analysis + Root Cause Identification + Response/Remediation

Project Architecture



Project Workflow



Team Members & Workload Distribution

Team Member	Major Responsibility
Muhammad Shayaan Ali	System architecture, Event Gateway & WebSockets, context builder, policy validator & executor, verification loop, ML integration, system integration, evaluation
Riddhima Sharma	LLM analysis engine (prompts, schema), YAML runbooks, LLM vs runbook testing, web dashboard, existing systems study, presentation
Shruti Tyagi	Monitoring & alert rules, demo app & fault injection, fault injector & load testing, ML model training, safety testing, report writing

Overall Work Distribution

- Frontend : Next.js, Tailwind CSS, Recharts – incident dashboard and approvals
- Backend : Python / FastAPI – Event Gateway, REST APIs, incident store (SQLite)
- Monitoring : Prometheus, Alertmanager, cAdvisor, redis_exporter
- Communication : WebSockets (incidents and response plans)
- AI / ML : Ollama (Llama 3.1 8B), prompt engineering; scikit-learn anomaly detector
- Response : Policy validator, Docker SDK executor, verifier
- Infrastructure : Docker Compose: frontend → api-service → redis
- Testing : pytest, promtool, Locust load, fault injector, safety suite

Meeting Details

Project Patent

Weekly Mentorship Meetings with Ms. Aliza Rana.

Date	Key Discussion / Work Done
19 Apr 2026 (Meeting 1)	Discussed the initial **project idea, problem statement and objectives** for the Intelligent DevOps Monitoring & Incident Response System.
22 May 2026 (Meeting 2)	Discussed the **overall system architecture and workflow**. Planned the use of **Prometheus for monitoring and metric collection**, with the collected data being passed to an intelligent analysis component for identifying DevOps issues.
01 Aug 2026 (Meeting 3)	Reviewed the project approach and discussed the creation of a **controlled DevOps test bench/testbed**.

Project Progress

Discussions with Xebia Industry Mentor Mr. Raygun Jose , Online Meetings on GoogleMeet

Date	Key Discussion Points
1 Aug 2026 (Meeting 1)	Discussed the project from an **industry-oriented perspective**, particularly the need for a realistic and repeatable **DevOps test bench**.
17 Aug 2026 (Meeting 2)	Discussed recreating optional problems, collecting metrics from Prometheus, passing the monitoring data to an LLM for incidental analysis, and developing a response/remediation mechanism

Project Planning / Timeline

Phase 1–2 · Setup & Monitoring ✔
- Research and architecture
- 3-service demo app (frontend → api → redis)
- Prometheus, cAdvisor, alert rules
- Fault injector and Locust load

Phase 3–4 · Gateway & LLM Analysis ✔
- Event Gateway and WebSockets
- Incident grouping, context builder
- LLM engine (Ollama, JSON schema)
- YAML runbook fallback

Phase 5–6 · Response & Verification ✔
- Policy validator and human approval
- Docker SDK executor
- Verifier, re-analysis (max 3), escalation
- Incident store (SQLite)

Phase 7–8 · ML Detector & Dashboard ✔
- Baseline data collection
- Isolation Forest + z-score detector
- Next.js dashboard
- Approvals and statistics

Phase 9–10 · Evaluation & Final (ongoing)
- 14 fault scenarios vs baselines
- Soak test for false alarms
- Report, demo video, paper
- Patent revision (Apr 2027)

Current Status & Results

All components are built and run together on Docker Compose. Evaluation stage 1: 9 fault scenarios × 3 runs on each system (81 runs, 11–12 Oct 2026).

Metric	B1: Rules + Runbooks	B2: Rules + LLM	S: Full System	Target
Faults detected	27/27	27/27	27/27	≥ 90%
False-positive incidents	0	0	0	≤ 10%
Mean time to detect	52 s	53 s	**27 s**	–
Root cause correct	100%	100%	100%	≥ 80%
Correct action executed	100%	96.3%	96.3%	≥ 85%
Unsafe actions executed	0	0	0	0
Auto-resolved	100%	100%	100%	≥ 60%
Mean time to resolve	75 s	102 s	**68 s**	–

- Safety: 29/29 adversarial (hallucinated or malicious) plans blocked; 0 unsafe actions in every run.
- ML detection: a slow memory leak was caught in 45 s, against 226 s with rules alone.
- Soak test: 10 hours with no faults gave 0 false alarms; 191 automated tests pass.
- Next: more evaluation rounds (10 and 20 runs per scenario), manual baseline (B0), harder scenarios.

Uniqueness of the Idea

What makes our system different?

Four elements, together in one closed loop:

- **Dependency-Aware Context Builder**
  Collects metrics from every dependent service around the incident, so the LLM can tell the root cause from a symptom.

- **Safety Validator between the LLM and the Executor**
  The LLM never runs anything: allowlisted actions, known targets, confidence ≥ 0.7, cooldown, human approval.

- **Re-analysis that Remembers Past Attempts**
  If verification fails, the actions already tried go back to the LLM; after 3 attempts it escalates.

- **Two Detectors, One Incident Format, Deterministic Fallback**
  Rules + ML feed the same incident; YAML runbooks take over if the LLM fails or is unsure.

Core Concept

Monitor → Detect → Analyze → Decide → Respond → Verify (→ loop)

Expected Outcome

At the end of the project, users should be able to:

- Monitor infrastructure in real time.
- View system health through a dashboard.
- Detect abnormal system behavior.
- Receive intelligent incident alerts.
- Analyze incidents using AI.
- Identify possible root causes.
- Approve or reject risky actions.
- Receive recommended remediation actions.
- Execute predefined automated responses.
- Track incident history and resolution.

Expected Benefits

- Reduced manual monitoring effort.
- Faster incident identification.
- Faster troubleshooting.
- Reduced response time.
- Improved system reliability.
- Better visibility into infrastructure health.
- More efficient DevOps operations.

Patent filing in progress

SHARDA UNIVERSITY
Beyond Boundaries

SHARDA UNIVERSITY

INVENTION DISCLOSURE FORM

CONFIDENTIAL

IMPORTANT

Please note that patent is always about a solution to an existing technical/industrial problem. As such, draft patent document (IDF) should be crafted in a manner so as to distinctly highlight the existing technical problem, the technical advancement proposed by the inventor(s), and the manner in which the proposed advancement solves the existing technical problem.

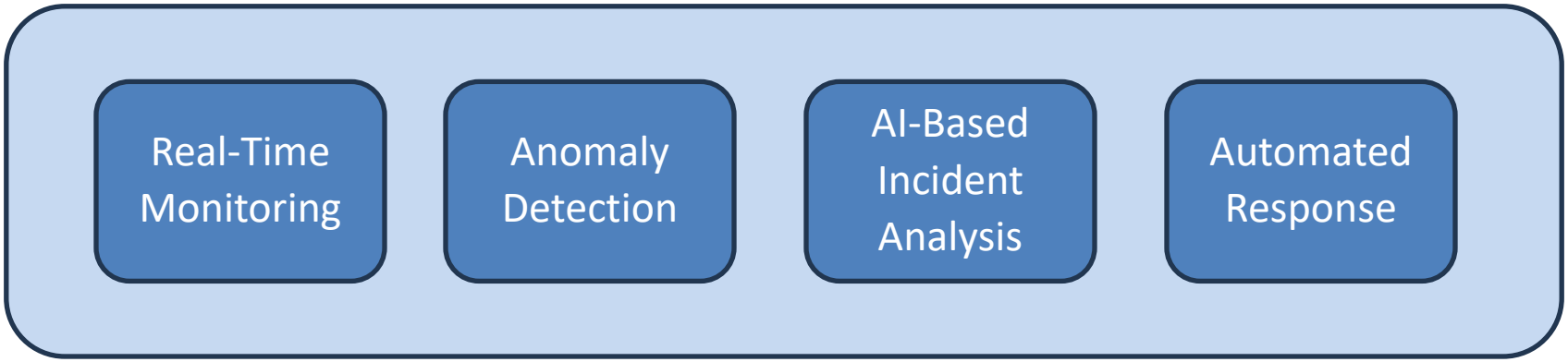
➢ **Particulars of Inventors**

S No	Name (Full)	Department / School	Designation	Mobile No.	Email	Official Address
1	Muhammad Shayaan Ali	Department of Computer Science and Engineering	Student	9136182311	2023433557.muhammad@ug.sharda.ac.in	Student,Greater Noida,Sharda University
2	Riddhima Sharma	Department of Computer Science and Engineering	Student	8860850003	2023567320.riddhima@ug.shardac.in	Student,Greater Noida,Sharda University
3	Shruti Tyagi	Department of Computer Science and Engineering	Student	8826043366	2023566902.shruti@ug.sharda.ac.in	Student,Greater Noida,Sharda University
4	Aliza Rana	Department of Computer Science and Engineering	Assistant Professor	9818306962	aliza.rana@sharda.ac.in	Assistant professor,Greater Noida,Sharda University

Conclusion

The Intelligent DevOps Monitoring & Incident Response System aims to transform traditional monitoring into an intelligent and automated operational process.

By combining:



The platform provides a unified approach to identifying, understanding and responding to infrastructure incidents.

The architecture is designed to be
Intelligent · Automated · Real-Time · Safe · Verifiable

Final Statement:
Our goal is not simply to build another monitoring dashboard, but to build an intelligent system that can understand incidents and assist in resolving them with minimal manual intervention.